

OMNIVERSEOS V1.0 RC

CYBERPUNK WEB OS

ARCHITECTURE & HISTORY

SUPER-DETAIL REPORT

OmniverseOS — Complete Engineering Report

Complete Architectural Blueprint, Feature Inventory, Testing Results & Historical Chronology from Day 1 to Today

REPORT MODE

Verified Read-Only

BACKEND SERVICES

FastAPI / Motor / LiteLLM

FRONTEND STACK

React 19 / Tailwind / Motion

AUDIT DATE

September 2026

29

PRODUCTION APPS

19

DESKTOP WIDGETS

105+

BACKEND FUNCTIONS

16

INDEXED COLLECTIONS

7

TTS AUDIO ENGINES

1,481

RESPONSIVE RULES

1. EXECUTIVE SUMMARY & ARCHITECTURAL IDENTITY

OmniverseOS is a cyberpunk-themed, browser-based desktop operating system combining an expansive multi-model AI suite, autonomous multi-agent chambers, a spatial vector memory engine, real-time voice intelligence, and a rich productivity and media ecosystem within a reactive window-managed interface.

Design Philosophy & Aesthetics: Built upon the *Electric & Neon* archetype. Deep void indigo backgrounds (`#05050A`), layered glassmorphic panels (`rgba(10,10,15,0.6)`), thin luminescent borders (`rgba(255,255,255,0.08)`), and vibrant functional semantic accents: AI Cyan (`#00F0FF`), Success Green (`#39FF14`), Danger Red (`#FF003C`), Warning Amber (`#F59E0B`), and Media Pink (`#F472B6`).

HIGH-LEVEL SYSTEM TOPOLOGY

BROWSER CLIENT

React 19 Shell · Framer Motion 11 · Radix UI · Sonner Toasts · Recharts · Lucide Icons
| Window Manager: Draggable / Resizable / Minimizable / Mobile Fullscreen Snap
| Cortex AI Core: Live Window Context Injection, Context Chips (D7), Session Isolation (D1)
| Multi-Engine Voice: Gemini 2.5 Flash, Fish Audio, Kokoro ONNX, Edge, Puter, Browser TTS
| 29 Lazy-Loaded Apps + 19 Desktop Widgets + Ambient Particle Canvas Layer

| /api/* (REST + Server-Sent Events)

FASTAPI BACKEND (:8001)

Lifespan Context Manager · 6 Modular Routers · Redis/Memory Sliding Window Rate Limiter
| Multi-Provider AI: Gemini 2.5, DeepSeek V3, Groq Llama-3.3, Cerebras, OpenRouter Fallback Chains
| Live Web Intelligence: TinyFish Web Crawl & Live Search Integration (web_service.py)
| Structured AI Engine: LiteLLM Router + Instructor Pydantic Validation (structured_ai.py)
| Hybrid Vector Service: text-embedding-004 + 128-dim Fallback Dense Vectorizer

| Async Motor Driver

MONGODB PERSISTENCE LAYER

16 Collections: users, notes, tasks, events, transactions, memories, files, images, clipboard, chat_messages, cortex_memories, memory_activity, chat_sessions, project_dna, decisions, timeline_events
Auto-Indexed on Startup: Compound indexes on (user_id, created_at), (user_id, session_id, created_at)

2. DETAILED CHRONOLOGICAL EVOLUTION: DAY 1 TO TODAY

Phase 0: Genesis & Day 1 Core OS MVP

The project originated as an ambitious initiative to converge modern computing paradigms—ChatGPT, Notion, Trello, Discord, Spotify, Google Calendar, YouTube, Netflix watchlists, virtual files, finance tracking, and in-browser code execution—into a single web operating system.

- **Desktop Window Engine:** Framer Motion draggable canvas with window minimization, cascade stacking, z-index elevation, active window focus rings, and custom desktop context menus.
- **JWT Auth Architecture:** Bcrypt hashing with salted passwords and 7-day HS256 JWT tokens.
- **Core Suite:** Initial releases of Notes, Kanban Tasks, Calendar, Finance, Files, Code Editor, Music, Videos, and Watchlist.

Phase 1: Staff Engineer Audit & System Stabilization (June 2026)

A rigorous engineering audit addressed critical security vulnerabilities, concurrency race conditions, and frontend memory leaks:

- **CORS & Credential Security:** Fixed browser silent rejection of wildcard origins with credentials. Replaced with regex origin validation (`allow_origin_regex=".*"`) when origins are wildcarded.
- **Unique Index & TOCTOU Fix:** Eliminated signup race conditions by creating an automated unique index on `users.email` in the startup lifespan.
- **Input Bounds & Abuse Prevention:** Enforced Pydantic `Field(max_length=...)` caps across prompt inputs and model whitelisting via `ALLOWED_MODELS`.
- **Voice.js Stale Closure Fix:** Resolved an issue where initial empty string closures caused the first spoken prompt to send empty text. Added mutable `transcriptRef` and double-start guards.
- **OSContext zCounter Functional Updates:** Switched to `setZCounter((z) => z + 1)` to prevent consecutive rapid app opens from receiving duplicate z-indexes.
- **AIChat Stream Abort Controllers:** Integrated `AbortController` and mount guards to cancel SSE fetch readers when closing the chat window mid-stream.

Phase 2: Alpha 0.9 RC & Cortex Unification Sprint (Late June 2026)

Executed a surgical, five-priority sprint unifying the system under Cortex awareness without changing UI branding:

- **P1 — Cortex Unification:** Wired `buildCortexSystemPrompt(osContext)` directly into AIChat. Cortex receives the active application, open window IDs, browser URL, and spatial memories on every LLM call. Unified browser URL tracking into `cortex_current_url` and propagated events through the activity timeline.
- **P2 — Workspace Restore:** Added `saveCurrentWorkspace(name)`, `restoreNamedWorkspace(name)`, and `restoreLastWorkspace()` to `OSContext`, clamping coordinates and restoring window hierarchies.
- **P3 — Universal Search (Spotlight Quality):** Overhauled `CommandPalette.js` (Ctrl+K) into a unified search across 9-10 distinct sources (Apps, Notes, Tasks, Memories, Timeline, Browser History, Clipboard, Snapshots, Quick Actions) with token scoring.
- **P4 — Mission Control Upgrade:** Added a 280px Cortex side rail featuring saved workspace layouts, recent 6 timeline events, clickable recent sites, and memory summaries.
- **P5 — Browser Intelligence:** Developed `BrowserIntelBar.js` to surface contextual actions for GitHub, YouTube, Stack Overflow, MDN, Reddit, Wikipedia, and Hacker News.
- **Deployment Ownership Audit:** Completed an exhaustive sweep confirming zero vendor runtime dependencies, enabling 100% self-hosted portability across Render, Vercel, and MongoDB Atlas.

Phase 3: Phase 1 Intelligence Layer Architecture

Expanded the backend data model and frontend capabilities with dedicated cognitive and workflow systems:

- **Conversation Archaeology:** Implemented `POST /api/ai/search/conversations` using regex search combined with Gemini reranking to surface prior discussion context.
- **Project DNA (`ProjectDNA.js`):** Created a comprehensive project workspace featuring Overview, Goals, Decisions, and Settings tabs connected to the `project_dna` collection.
- **Decision Memory:** Structured logging of architectural and strategic decisions with rationale and status tracking (`decisions` collection).
- **Cortex Timeline (`TimelineApp.js`):** Day-grouped timeline merging local browser activity with persistent server events.
- **Cortex Interrupts (`CortexInterrupts.js`):** Ambient check-in engine querying Gemini Flash Lite every 8 minutes with auto-dismissal and focus-mode muting.
- **Focus Tunnel (`FocusTunnel.js`):** Distraction-free work portal with pre-focus goal selection, countdown timer, and post-session Gemini reflection.
- **Ghost Writer & Morning Briefing:** Integrated inline ghost text autocompletion in Notes (`useGhostWriter.js`) and return catch-up briefings (`MorningBriefing.js`).
- **Dedicated Intelligence Client:** Isolated intelligence methods into `frontend/src/lib/intelligenceApi.js` to keep `api.js` focused.

Phase 4: Destination Features & High-Level Cognitive Suite

Engineered an advanced suite of high-impact AI analytical tools designed for strategic thinking, stress-testing, and synthesis:

- **The Adversary (`Adversary.js`):** Aggressive challenge engine adopting ruthless personas (VC, Engineer, Competitor) to probe flaws in logic or product proposals.
- **War Room (`WarRoom.js`):** 5-agent strategic debate chamber with animated responses from The Investor, The Customer, The Competitor, The Internal Critic, and The Journalist.
- **Dead Reckoning (`DeadReckoning.js`):** Trajectory projection evaluating 1-year, 3-year, and 5-year outcomes against user actions and stated goals.
- **Neural Matrix (`NeuralMatrix.js`):** Real-time 3D spatial constellation HUD visualizing vector memory nodes, active AI agents, and system health metrics.
- **Omniverse Mirror (`OmniverseMirror.js`):** Digital twin simulating parallel lives, cognitive shadows, decision time travel, and personal counterfactuals.
- **Omniverse Zero (`OmniverseZero.js`):** Deep problem analysis suite with a Possibility Collider, Problem Time Machine, and Final Boss Audit.
- **The Black Box (`BlackBoxApp.js`):** Multi-phase cognitive anatomy framework diagnosing hidden centers of gravity in complex decisions.
- **Model Face-Off & Swarm Goal:** Side-by-side comparative model debate and autonomous multi-agent task breakdown.

Phase 5: Mobile & Tablet Responsive Overhaul

Re-architected the OS to deliver an authentic native-feeling experience across phone and tablet viewports without degrading desktop layouts:

- **Adaptive Geometry (`useBreakpoint.js`):** Viewport categorization into Mobile (<768px), Tablet (768–1023px), and Desktop (≥ 1024 px).
- **Mobile Fullscreen Snapping:** Framer Motion geometry overrides window coordinates on mobile to fill the viewport between the 56px top bar and 80px dock, disabling window dragging.
- **CSS Responsive Layer (`responsive.css` - 1,481 lines):** Uses `data-testid` selectors to enforce touch targets (≥ 36 px), scroll containment, safe-area insets, and iOS font zoom prevention (16px base font size).
- **Mobile Shell Components:** Built `MobileHomeScreen.js`, `MobileAppDrawer.js`, `MobileWidgetView.js`, and `AdaptiveDock.js`.

Phase 6: Multi-Engine Voice & Audio Architecture

- **7-Engine TTS Pipeline** (`cortexTTSManager.js`): Synthesis through Gemini 2.5 Flash, Fish Audio, Kokoro ONNX, Edge TTS, Puter TTS, Stream TTS, and Web Speech API.
- **TTS In-Memory Caching**: SHA-256 keyed cache (`_tts_cache`) preventing redundant network requests for identical speech synthesis phrases.
- **Interruption & Recognition**: Real-time STT with regional BCP-47 language support (en-US, en-IN, en-GB, etc.), emotion prosody modulation, and instant speech interruption.

Phase 7: Live Web Intelligence & Structured Extraction

- **TinyFish Live Web Search** (`web_service.py`): Single gateway for live web crawling with automated search intent heuristics and confidence scoring.
- **LiteLLM + Instructor Extraction** (`structured_ai.py`): Pydantic schema validation ensuring raw unvalidated LLM output is rejected before database insertion.
- **Hybrid Vector Engine** (`core/vector_service.py`): Dual-mode embeddings using Gemini `text-embedding-004` with automatic fallback to a 128-dimensional normalized deterministic vectorizer.

Phase 8: RC1 Hardening, Context Chips & Security Remediation (August 2026)

- **D7 Context Chips** (`ContextChips.js`): Pill chip row rendered above the AI Chat prompt displaying live sources (Active App, Open Apps, Browser URL, Memories) with instant click-to-exclude toggling.
- **D1 Chat Session Isolation**: Session CRUD with auto-titling, pinning, duplication, and strict per-session message isolation.
- **Security Incident Remediation**: Redacted leaked LLM key at HEAD, established `SECURITY_INCIDENT.md`, and hardened `.env` parsing.
- **LocationSetup Modal Hardening**: Corrected pointer-event interception on modal backdrop; added dedicated backdrop button, Escape key listener, and skip flow.

3. COMPLETE PRODUCTION APPLICATION CATALOG (29 APPS)

Every application in OmniverseOS is registered in `frontend/src/lib/apps.js` and loaded via `React.lazy()` for code splitting and instant desktop initialization.

APP ID	NAME	GROUP	ACCENT	KEY FEATURES & TECHNICAL CAPABILITIES
dashboard	Dashboard	Core	#00F0FF	System health HUD, quick launch tiles, real-time clock, telemetry gauges, and customizable bento grid.
chat	AI Chat	AI	#00F0FF	Multi-provider SSE streaming, Context Chips (D7), session management (D1), model debate, and command execution.
image	Image Gen	AI	#A855F7	Prompt-driven image synthesis via Gemini Imagen / OpenAI gateway with history gallery and aspect ratio controls.
voice	Cortex Voice	AI	#4A9EFF	Hands-free voice assistant with 7 TTS engines, real-time STT, speech interruption, and CyberOrb visualizer.
memory	Memory	AI	#2DD4BF	Spatial vector memory explorer with category categorization, importance scoring, and never-forget toggles.
projects	Project DNA	AI	#00F0FF	Project architecture tracker with Overview, Goals, Decisions, Settings, and AI context extraction.
timeline	Timeline	AI	#7B2FFF	Chronological event stream capturing navigation, app launches, voice commands, and system mutations.
notes	Notes	Productivity	#F59E0B	Rich-text markdown note-taking with color coding, search filtering, and Ghost Writer autocompletion.
tasks	Tasks	Productivity	#39FF14	Drag-and-drop Kanban task board with Todo, In Progress, and Done lanes plus priority tags.
calendar	Calendar	Productivity	#FB923C	Month/Week/Agenda scheduler with responsive header formatting and compact event indicators on mobile.
clipboard	Clipboard	Productivity	#818CF8	Persistent clipboard history manager with quick copy-to-clipboard actions, search, and item pinning.
music	Music	Media	#F472B6	Cyberpunk music player with ambient playlist streaming, track visualizer, and desktop mini-widget sync.
videos	Videos	Media	#F472B6	Curated video player supporting categorized YouTube content and embedded theater-mode playback.
watchlist	Watchlist	Media	#F472B6	Personal media tracking watchlist for films, documentaries, and series with rating tags.
files	Files	System	#60A5FA	Virtual filesystem explorer managing folders, uploaded assets, code snippets, and storage telemetry.
code	Code Editor	System	#39FF14	Syntax-highlighted JavaScript editor with console output and isolated <code>new Function()</code> execution.
browser	Browser	System	#60A5FA	Iframe web browser with bookmark shortcuts, history logging, and contextual BrowserIntelBar AI prompts.
settings	Settings	System	#94A3B8	OS personalization: dynamic wallpapers, sound FX volumes, API key entry, location preferences.
finance	Finance	Data	#39FF14	Financial tracker with monthly expense breakdowns, balance trends, Recharts graphs, and transaction logging.
analytics	Analytics	Data	#39FF14	System metrics dashboard visualizing app usage frequency, activity heatmaps, and productivity scores.
nebula	Nebula Chat	Social	#A855F7	Cyberpunk Discord-style communications terminal with simulated team channels and active user statuses.
swarm	Swarm Goal	AI	#00F0FF	Multi-agent goal execution suite decomposing high-level targets into parallel sub-agent workflows.
faceoff	Model Face-Off	AI	#00F0FF	Side-by-side model debate comparing Gemini, DeepSeek, Llama, and Cerebras outputs simultaneously.
adversary	The Adversary	AI	#FF003C	Adversarial challenge chamber stress-testing ideas with relentless critique from customized personas.
warroom	War Room	AI	#F59E0B	Strategic multi-persona council (Investor, Customer, Competitor, Critic, Journalist) with animated text.
deadreckoning	Dead Reckoning	AI	#7B2FFF	Long-term behavioral trajectory projector analyzing 1/3/5-year horizons and critical leverage points.

APP ID	NAME	GROUP	ACCENT	KEY FEATURES & TECHNICAL CAPABILITIES
matrix	Neural Matrix	AI	#00F0FF	Spatial 3D interactive constellation HUD linking vector memories, active agents, and system nodes.
mirror	Omniverse Mirror	AI	#A855F7	Digital twin simulator featuring cognitive shadow analysis, parallel timelines, and counterfactual memory.
zero	Omniverse Zero	AI	#00F0FF	Cognitive problem collider, timeline rewind engine, and final boss audit for high-stakes problem spaces.

4. DESKTOP WIDGET SYSTEM (19 PRODUCTION WIDGETS)

OmniverseOS features an extensible widget canvas (`WidgetCanvas.js`) rendered on the desktop beneath floating windows. Users can add, reposition, and configure widgets from the integrated Widget Store.

Widget Name	Component File	Operational Role & Capabilities
AI Command Center	<code>widgets/AICommandCenter.js</code>	Desktop natural language input bar triggering immediate Cortex actions, queries, or app launches.
Calendar Widget	<code>widgets/CalendarWidget.js</code>	Compact month calendar showing today's date and upcoming scheduled events with status indicators.
Chrono Widget	<code>widgets/ChronoWidget.js</code>	Multi-mode precision stopwatch, lap timer, countdown clock, and international timezone tracker.
Clock Widget	<code>widgets/ClockWidget.js</code>	Minimalist neon digital desktop clock with dynamic second ticks and locale-formatted date string.
Cortex Living Widget	<code>widgets/CortexLivingWidget.js</code>	Animated reactive AI avatar displaying Cortex's thinking, listening, or idle status in real time.
Cortex Smart Widget	<code>widgets/CortexSmartWidget.js</code>	Contextual intelligence card suggesting next actions based on currently open windows and recent URLs.
Cortex Mini Widget	<code>widgets/CortexWidget.js</code>	One-click audio activation pill providing instant voice input from the desktop background.
Focus Timer Widget	<code>widgets/FocusTimerWidget.js</code>	Pomodoro timer with radial progress bar, interval presets, and notification integration.
Music Mini Widget	<code>widgets/MusicWidget.js</code>	Desktop audio controls displaying track title, artist, audio scrub bar, and play/pause/skip triggers.
Neural Wallpaper	<code>widgets/NeuralWallpaper.js</code>	Interactive canvas wallpaper generating reactive particle constellations following cursor movement.
News Feed Widget	<code>widgets/NewsWidget.js</code>	Live scrolling headline ticker streaming tech news, AI developments, and global updates.
Plasma Widget	<code>widgets/PlasmaWidget.js</code>	Generative WebGL/canvas shader rendering soothing cyberpunk plasma fluid animations.
Quick Notes Widget	<code>widgets/QuickNotesWidget.js</code>	Zero-latency desktop scratchpad automatically syncing quick memos to the main Notes database.
Recent Activity	<code>widgets/RecentActivityWidget.js</code>	Live timeline card listing the last 5 operating system actions, app navigations, or voice commands.
System Monitor	<code>widgets/SystemMonitorWidget.js</code>	Visual resource telemetry displaying simulated CPU, memory pressure, and network throughput graphs.
System Status	<code>widgets/SystemStatusWidget.js</code>	Real-time backend API connection status, MongoDB ping latency, and rate-limiter capacity meter.
Todo Widget	<code>widgets/TodoWidget.js</code>	Quick-access checklist allowing users to check off tasks directly from the desktop canvas.
Weather Widget	<code>widgets/WeatherWidget.js</code>	Current weather conditions, temperature, humidity, and forecasts based on stored location settings.
World Clock	<code>widgets/WorldClockWidget.js</code>	Multi-city international clock grid displaying UTC, Tokyo, London, New York, and San Francisco.

5. BACKEND ARCHITECTURE & DATABASE ENGINE

Core Router Architecture

The backend is engineered around a central FastAPI service in `backend/server.py` (122 KB, 2,863 lines, 105+ functions) combined with 6 specialized sub-routers in `backend/routers/`:

`routers/tts.py`

Handles Gemini 2.5 Flash and Fish Audio TTS synthesis with an in-memory SHA-256 LRU cache (`_tts_cache`), concurrency locks, and audio format streaming.

`routers/auth.py`

User registration, credential login, profile retrieval, and password updates with bcrypt hashing and JWT token issuance.

`routers/productivity.py`

Full CRUD operations for notes, tasks, events, transactions, virtual files, and multi-entry clipboard history.

`routers/memory.py`

Spatial memory engine handling vector storage, category filtering, importance ranking, and cosine similarity queries.

`routers/agents.py`

Dedicated SSE streaming endpoints for The Adversary, War Room debate council, Dead Reckoning, and Model Face-Off.

`routers/system.py`

System telemetry endpoint reporting MongoDB latency, collection document counts, and rate limiter status.

MongoDB Schema & Startup Index Automation

During the FastAPI `lifespan` startup hook, the system automatically verifies and creates compound and unique indexes across all 16 database collections:

COLLECTION NAME	INDEXES ENFORCED ON STARTUP	PURPOSE / ACCESS PATTERN
<code>users</code>	<code>email</code> (unique)	Prevents duplicate account registrations and TOCTOU races.
<code>notes</code> , <code>tasks</code> , <code>events</code> , <code>transactions</code> , <code>memories</code> , <code>files</code> , <code>images</code> , <code>clipboard</code>	Compound: <code>(user_id, 1)</code> , <code>(created_at, -1)</code>	Eliminates full collection scans on per-user reverse-chronological list queries.
<code>chat_messages</code>	Compound: <code>(user_id, 1)</code> , <code>(session_id, 1)</code> , <code>(created_at, 1)</code>	High-speed session conversation history retrieval and message ordering.
<code>cortex_memories</code>	<code>(user_id, 1)</code> , <code>(importance_score, -1)</code> <code>(user_id, 1)</code> , <code>(category, 1)</code> <code>(user_id, 1)</code> , <code>(never_forget, 1)</code>	Fast retrieval of high-priority memories for live Cortex system prompt injection.
<code>chat_sessions</code>	<code>id</code> (unique) Compound: <code>(user_id, 1)</code> , <code>(pinned, -1)</code> , <code>(updated_at, -1)</code>	Ensures pinned sessions appear first in the sidebar followed by recent activity.
<code>project_dna</code>	<code>id</code> (unique), Compound: <code>(user_id, 1)</code> , <code>(updated_at, -1)</code>	Project architecture retrieval and updates.
<code>decisions</code>	<code>id</code> (unique), Compound: <code>(user_id, 1)</code> , <code>(project_id, 1)</code>	Decision memory tracking per project.
<code>timeline_events</code>	<code>id</code> (unique), Compound: <code>(user_id, 1)</code> , <code>(created_at, -1)</code>	Activity timeline stream queries.

Rate Limiter Architecture (`rate_limiter.py`)

Enforces sliding-window rate limits across AI endpoints (30/min/user for chat, 8/min/user for images, 10/min/user for agent tools). Employs an abstract base class supporting:

- Production Redis Pipeline:** Atomic sliding-window enforcement using Redis sorted sets (`zremrangebyscore`, `zcard`, `zadd`, `expire`).
- Development Memory Fallback:** Sliding-window queue with monotonic clock timestamps and an `asyncio.Lock` mutex for single-worker or local dev environments.

6. TESTING RESULTS, QA SWEEPS & SECURITY POSTURE

Quality Verification & Regression Test Suite

OmniverseOS maintains rigorous testing procedures documented in `test_reports/` and `test_result.md`:

- **Backend Pytest Suite** (`backend/tests/test_backend.py`): 13 automated tests verifying health checks, auth signup/login/profile, full CRUD across notes, tasks, events, transactions, memories, clipboard, and files, plus analytics summaries. Executes in ~1.5 seconds.
- **E2E Regression Runner** (`backend_test.py`): Comprehensive automated probe testing live session creation, auto-titling, SSE streaming, memory relevance searches, and error handling.
- **Iteration 1 & 2 Results:** Confirmed 100% pass rates on backend functional tests and verified end-to-end user journeys from auth to desktop, Command Palette, and Mission Control.

Security Incident & Historical Remediation

Security Audit (August 2026): An automated test run had inadvertently logged a legacy universal LLM proxy key into `test_result.md`. Upon identification:

1. The credential was immediately redacted from HEAD in commit `4d9b631`.
2. A comprehensive incident report was authored and committed in `SECURITY_INCIDENT.md` detailing the scope and providing key rotation instructions.
3. Preventive controls were established to ensure test logging scripts record environment variable names only, never literal secret values.
4. A repository-wide scan confirmed no other credentials, private keys, or tokens exist in current files.

Self-Hosted Deployment Readiness

The codebase is 100% portable and free of vendor lock-in:

- **Docker Compose:** Single command deployment (`docker compose up -d`) provisioning MongoDB 7, FastAPI backend on `:8001`, and React frontend on `:3000`.
- **Render & Vercel:** Production ready by deploying the `backend/` folder to Render Web Services and `frontend/` to Vercel with `REACT_APP_BACKEND_URL` pointing to the Render endpoint.
- **Graceful Degradation:** If external AI provider keys are omitted, the operating system boots normally and all OS features (window manager, notes, tasks, calendar, files, finance, settings) remain fully functional.

7. CURRENT STATUS & FUTURE ROADMAP

Current Status: The operating system is fully built, highly feature-rich, and architecturally stabilized. All 29 applications and 19 widgets are functional. The codebase represents a complete, cohesive vision of an AI-first operating system.

Future Roadmap (Post-RC1)

- **Electron Desktop Migration:** Full blueprint documented in `memory/ELECTRON_MIGRATION_BLUEPRINT.md`. Packaging OmniverseOS as a native macOS/Windows desktop application using Chrome DevTools Protocol (CDP), Playwright automation, and native OS notifications.
- **Frontend RTL/Jest Test Suite:** Expand automated unit test coverage to achieve $\geq 90\%$ branch coverage across frontend components.
- **Collaborative Workspaces:** Real-time CRDT synchronization for multi-user editing in Notes, Tasks, and War Room debate sessions.